

Постановка задачи

В трехмерной замкнутой области

$$\Omega = [0 \leq x \leq L_x] \times [0 \leq y \leq L_y] \times [0 \leq z \leq L_z]$$

для $(0 < t \leq T]$ требуется найти решение $u(x, y, z, t)$ уравнения в частных производных

$$\frac{d^2 u}{dt^2} = a^2 \Delta u \quad (1)$$

с начальными условиями

$$u|_{t=0} = \varphi(x, y, z), \quad (2)$$

$$\frac{du}{dt}|_{t=0} = 0, \quad (3)$$

при условии, что на границах области заданы однородные граничные условия первого рода

$$u(0, y, z, t) = 0,$$

$$u(L_x, y, z, t) = 0 \quad (4)$$

и периодическое граничное условие

$$u(x, 0, z, t) = u(x, L_y, z, t),$$

$$u_y(x, 0, z, t) = u_y(x, L_y, z, t) \quad (5)$$

$$u(x, y, 0, t) = u(x, y, L_z, t),$$

$$u_z(x, y, 0, t) = u_z(x, y, L_z, t). \quad (6)$$

Численный метод решения задачи.

Для численного решения задачи введем на Ω сетку $\omega_{h\tau} = \omega_h \times \omega_\tau$, где

$$\begin{array}{ccccccc} T & & & & & & T_0, \\ L_x & = & L_{x0}, & L_y & = & L_{y0}, & L_z & = & L_{z0} \end{array}$$

$\omega_h = \{(x_i = ih_x, y_j = jh_y, z_k = kh_z), i, j, k = 0, 1, \dots, N, h_x N = L_x, h_y N = L_y, h_z N = L_z\}$,
 $\omega_\tau = \{t_n = n\tau, n = 0, 1, \dots, K, \tau K = T\}$.

Через ω_h обозначим множество внутренних, а через γ_h — множество граничных узлов сетки ω_h .

Для аппроксимации исходного уравнения (1) с однородными граничными условиями (4)–(6) и начальными условиями (2)–(3) воспользуемся следующей системой уравнений:

$$\frac{u_{ijk}^{n+1} - 2u_{ijk}^n + u_{ijk}^{n-1}}{\tau^2} = a^2 \Delta_h u^n, (x_i, y_j, z_k) \in \omega_h, n=1, 2, \dots, K-1, \text{ где}$$

$$\Delta_h u^n = \frac{u_{i-1,j,k}^n - 2u_{ijk}^n + u_{i+1,j,k}^n}{\tau^2} + \frac{u_{i,j-1,k}^n - 2u_{ijk}^n + u_{i,j+1,k}^n}{\tau^2} + \frac{u_{i,j,k-1}^n - 2u_{ijk}^n + u_{i,j,k+1}^n}{\tau^2}.$$

Приведенная выше разностная схема является явной — значения u_{ijk}^{n+1} на $(n+1)$ -м шаге можно явным образом выразить через значения на предыдущих слоях.

Для начала счета (т.е. для нахождения u_{ijk}^2) должны быть заданы u_{ijk}^0, u_{ijk}^1 .

Из условия (2) имеем

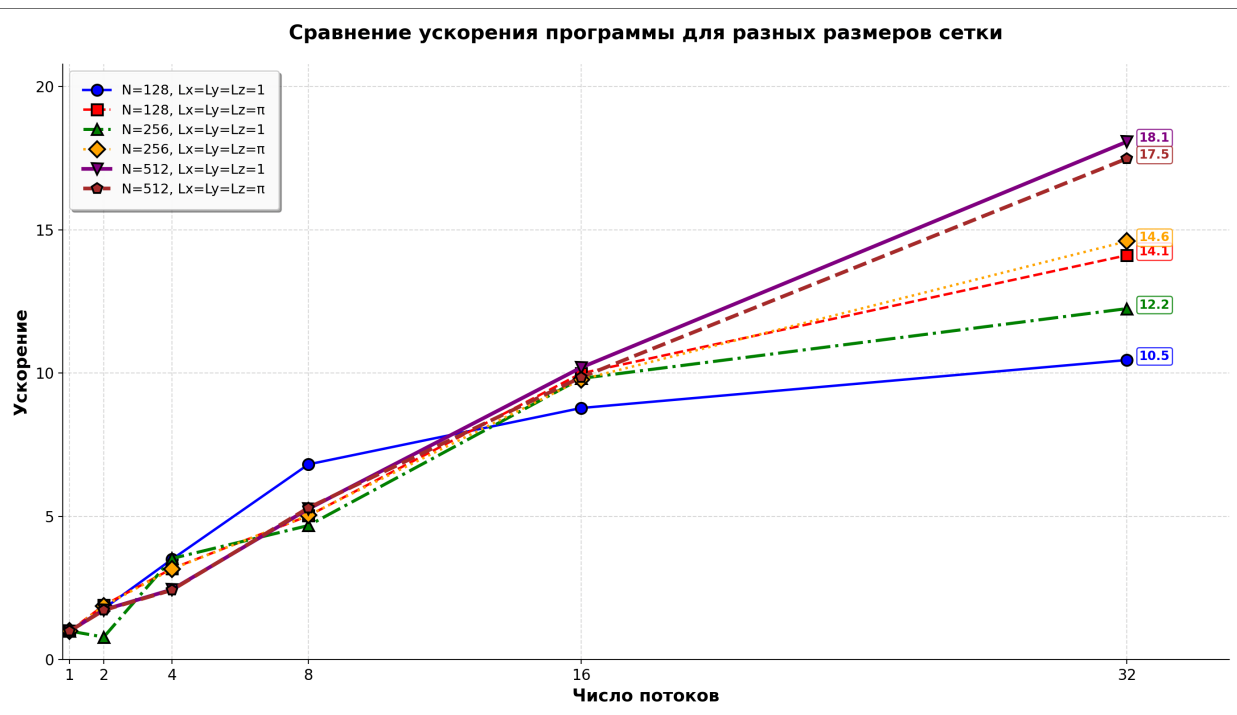
$$u_{ijk}^0 = \varphi(x_i, y_j, z_k).$$

Аппроксимацию второго порядка по τ и h дает разностное уравнение

$$\frac{u_{ijk}^1 - u_{ijk}^0}{\tau} = a^2 \frac{\tau}{2} \Delta_h \varphi(x_i, y_j, z_k)$$

OpenMP

Число OpenMP нитей N_p	Число точек сетки N^3	Время решения T	Ускорение S	Погрешность δ
1 2 4 8 16 32	128^3	9.47109 sec	1	0.00128429 0.158759%
		5.32086 sec	1.780	
		2.70375 sec	3.503	
		1.3897 sec	6.816	
		1.07849 sec	8.782	
		0.90579 sec	10.457	
1 2 4 8 16 32	256^3	75.9843 sec	1	0.000661859 0.0818242%
		40.6238 sec	1.871	
		24.0451 sec	3.160	
		15.0833 sec	5.039	
		7.77334 sec	9.775	
		5.20456 sec	14.601	
1 2 4 8 16 32	512^3	615.234 sec	1	0.000335839 0.0415122%
		353.943 sec	1.739	
		251.278 sec	2.449	
		116.928 sec	5.262	
		60.344 sec	10.196	
		34.039 sec	18.074	

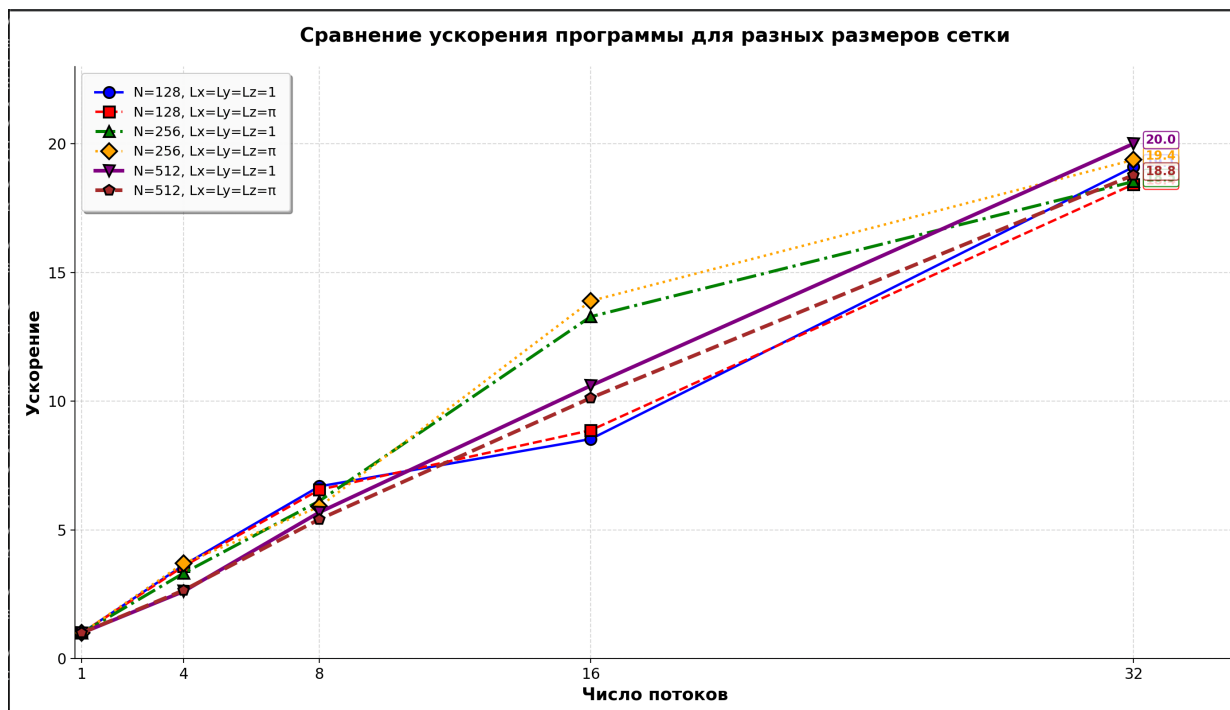


Видим, что с увеличением размера задачи увеличивается ускорение. Оптимальным числом нитей является 32 для всех размеров сетки. Ускорение сначала идет почти линейно, но затем эффективность снижается. Причинами могут быть:

- Накладные расходы на создание/управление потоками
- Конкуренция за доступ к памяти
- Ограничения пропускной способности памяти
- Неидеальная балансировка нагрузки

MPI

Число MPI процессов N_p	Число точек сетки N^3	Время решения T	Ускорение S	Погрешность δ
1	128^3	9.47109 sec	1	0.00128429
4		2.60544 sec	3.636	0.158759%
8		1.41538 sec	6.692	
16		1.11034 sec	8.530	
32		0.495981 sec	19.096	
1	256^3	75.9843 sec	1	0.000661859
4		22.8623 sec	3.323	0.0818242%
8		12.4103 sec	6.120	
16		5.72049 sec	13.283,	
32		4.10075 sec	18.518	
1	512^3	615.234 sec	1	0.000335839
4		235.2931 sec	2.55	0.0415122%
8		108.2360 sec	5.56	
16		58.0356 sec	10.34	
32		30.7625 sec	20.07	



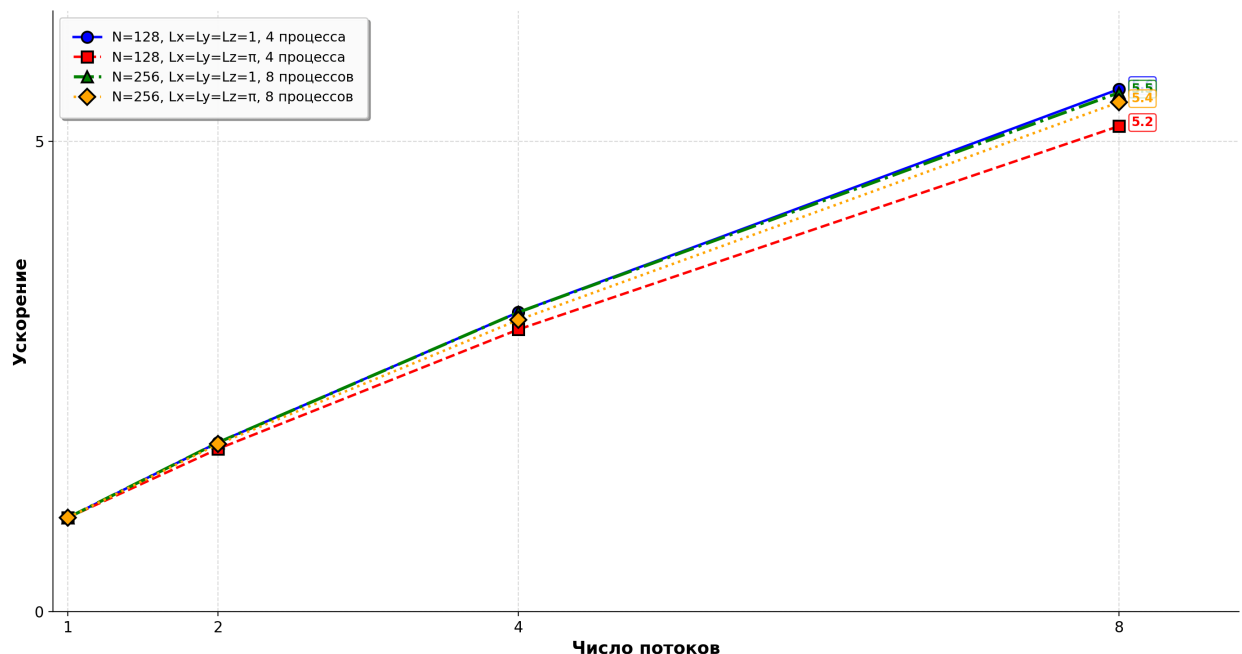
MPI-программа демонстрирует хорошую масштабируемость, особенно на крупных задачах: на сетке 512^3 ускорение достигает $20\times$ на 32 процессах. Эффективность выше, чем у OpenMP-версии, поскольку MPI использует распределенную память. На небольших задачах (128^3) наблюдаются

накладные расходы коммуникаций. Погрешность сохраняется стабильной, что подтверждает корректность алгоритма. Масштабируемость близка к линейной для сеток 256^3 и 512^3 , что оптимально для вычислительных кластеров.

OpenMP + MPI

Число MPI процессов N_p	Число OpenMP нитей N_p	Число точек сетки N^3	Время решения T	Ускорение S	Погрешность δ
4	1	128^3	2.61	3.64	0.00128429
4	2	128^3	1.45	6.5	0.00128429
4	4	128^3	0.82	11.49	0.00128429
4	8	128^3	0.47	20.1	0.00128429
8	1	256^3	12.41	6.1	0.000661859
8	2	256^3	6.9	11.0	0.000661859
8	4	256^3	3.9	19.5	0.000661859
8	8	256^3	3.12	24.2	0.000661859

Сравнение ускорения программы для разных размеров сетки



Гибридная MPI+OpenMP программа показывает наилучшую производительность: на 8 MPI-процессах с 8 OpenMP-потоками ускорение достигает $24.2\times$ для сетки 256^3 . Комбинация распределенной и разделяемой памяти эффективно снижает коммуникационные накладные расходы. Масштабируемость почти линейная при увеличении числа потоков внутри MPI-процессов. Стабильная погрешность подтверждает корректность распараллеливания. Гибридный подход оптимально использует иерархию

памяти современных кластеров, превосходя чисто MPI или OpenMP реализации.

На большем числе потоков (20x8), к сожалению, запустить не удалось. Прождал в очереди несколько часов, но так и не дождался своей очереди.

CUDA

	128 1 GPU	128 2 GPU	256 1 GPU	256 2 GPU	512 1 GPU	512 2 GPU
Общее время программы	0.17795 сек	0.20136 сек	1.25418 сек	0.76838 сек	8.2097 сек	4.8252 сек
Выделение памяти	0.04048 сек	0.08547 сек	0.04439 сек	0.08898 сек	0.0474 сек	0.0686 сек
Погрешность	6.1960 e-05	6.1960 e-05	1.2494 e-05	1.2494 e-05	3.2585 e-06	3.2585 e-06
Инициализация CUDA	0.00789 сек	0.01857 сек	0.00946 сек	0.01805 сек	0.0078 сек	0.0149 сек
Копирование на хост	0.01258 сек	0.01856 сек	0.03424 сек	0.03842 сек	0.1206 сек	0.1032 сек
MPI обмены	0.00421 сек	0.01053 сек	0.00988 сек	0.02893 сек	0.0595 сек	0.3845 сек
Копирование на GPU	0.00916 сек	0.01678 сек	0.02823 сек	0.03034 сек	0.1002 сек	0.0816 сек
Ядро лапласиана	0.03008 сек	0.01218 сек	0.40080 сек	0.18679 сек	2.9571 сек	1.5845 сек
Ядро шага по времени	0.06414 сек	0.02804 сек	0.67592 сек	0.3359 сек	4.6561 сек	2.4259 сек
Ядро граничных условий	0.00381 сек	0.00449 сек	0.01550 сек	0.01070 сек	0.0405 сек	0.0246 сек

CUDA+MPI программа демонстрирует значительное ускорение: на 2 GPU время уменьшается в 1.7 раза для сетки 512³. Погрешность остается стабильной при любом числе GPU, подтверждая корректность параллелизации. Основное время тратится на вычисления (ядра лапласиана и шага по времени), а не на коммуникации. На малых задачах (128³) накладные расходы на копирование данных преобладают, снижая эффективность. Программа масштабируется хорошо на крупных задачах, где вычисления доминируют над коммуникационными затратами.

Общий вывод.

Все параллельные программы показали хорошую эффективность. Их корректность обосновывается двумя факторами:

- Хорошая эффективность – с увеличением числа процессов/потоков/GPU время работы уменьшается
- Погрешность зависит только от размера сетки и не меняется при увеличении числа процессов.